CraftCV Frontend–Backend Integration Guide

Reusable Master Data + CV-Specific Selection

1. Purpose

The CraftCV backend is designed so that a user's personal information can be stored as **reusable master data**.

A user should not have to retype the same education, work experience, skills, certifications, languages, awards, or additional information every time they create a new CV.

The frontend should therefore integrate with the backend using two main concepts:

- **Master Data** — reusable information owned by the user.
- **CV Data** — a specific CV that selects which master records should appear in that CV.

Core idea

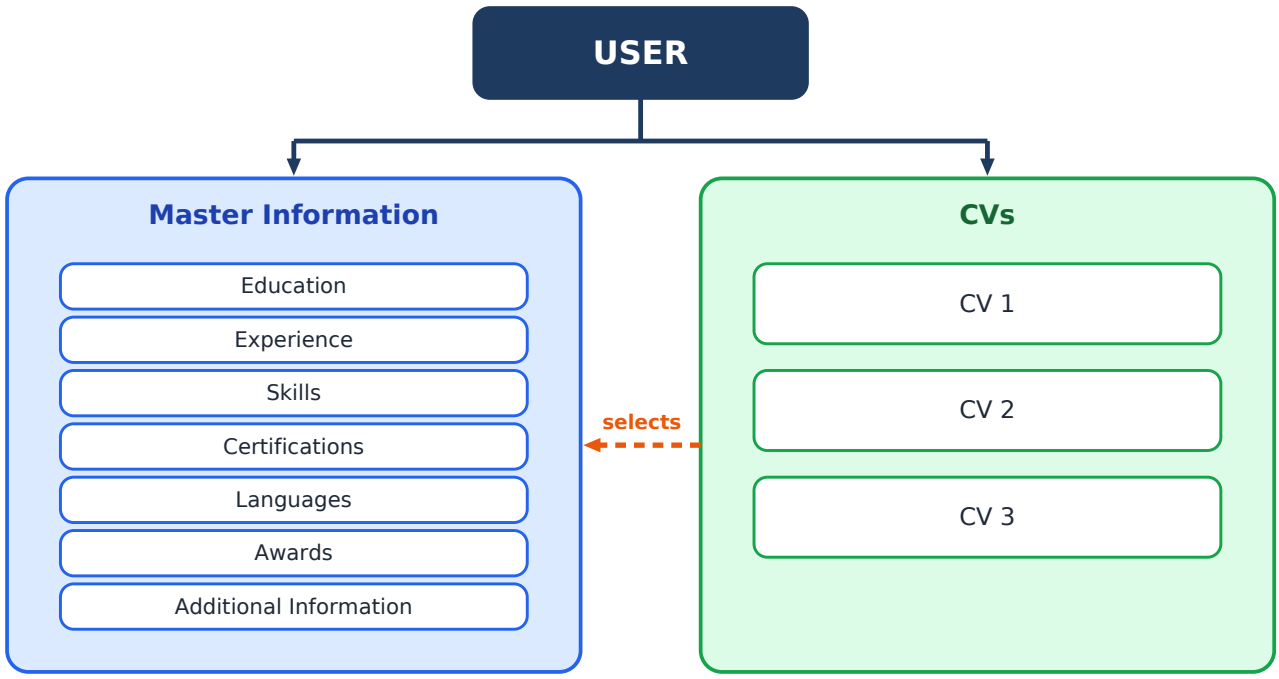


Figure 1. Each CV selects the master information that is relevant to it.

2. How the CV Editor Should Work

The CV Editor should **not treat every new CV as a completely empty form**. Instead, when the user opens a section such as Education, the frontend should show the user's existing master records.

Example

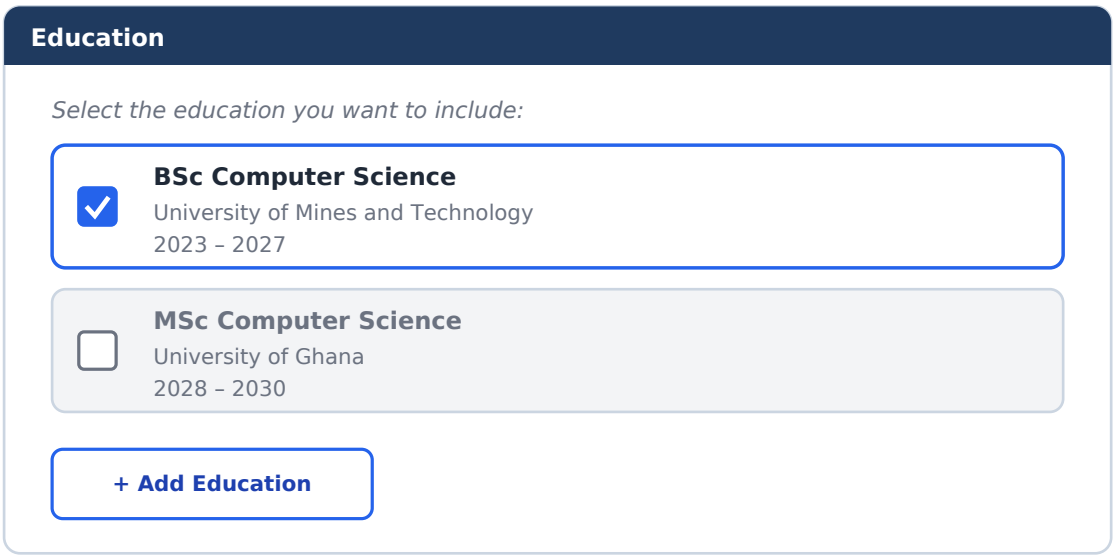


Figure 2. Education section of the CV Editor, showing existing master records.

The user should be able to:

- Select an existing record.
- Deselect an existing record.
- Add a new record.
- Edit an existing record.
- Remove a record from the current CV.

3. Important: Remove From CV ≠ Delete Master Data

This is one of the most important parts of the integration.

Suppose the user has two master education records, *BSc Computer Science* and *MSc Computer Science*. The user creates **CV #1** and selects BSc. Later, they remove BSc from CV #1.

> The frontend should **not delete the Education record**. It should only remove the relationship between CV #1 and BSc Computer Science. The master record should remain available.

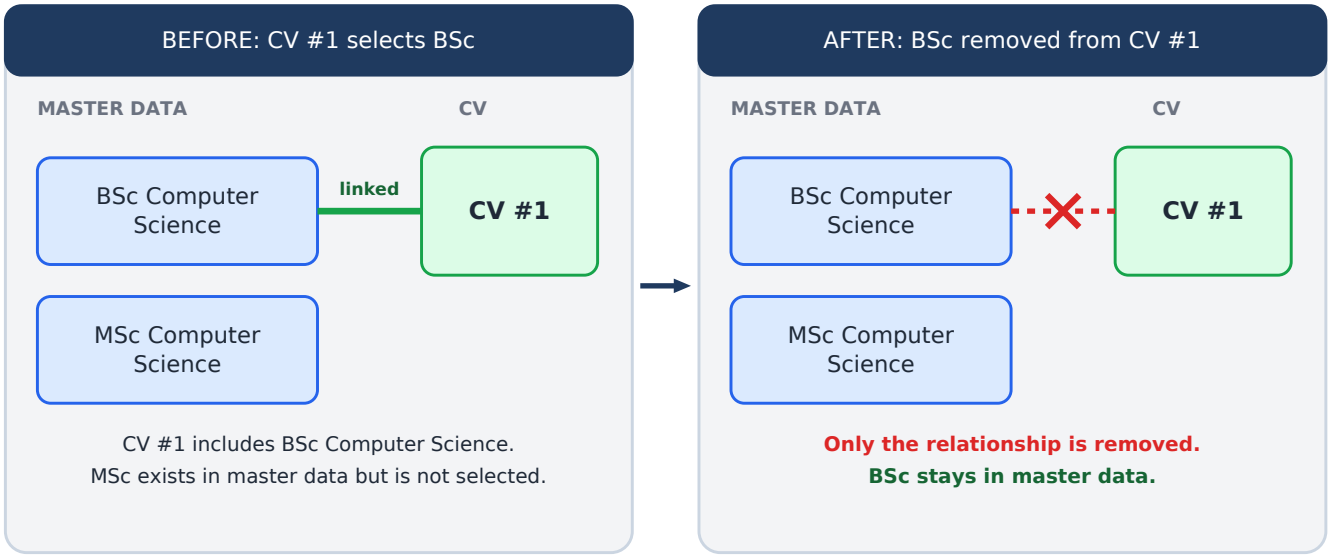


Figure 3. Removing a record from a CV removes only the link, never the master record.

When the user creates CV #2, the Education section shows both records unchecked, and BSc is still available to select:

```
Education

□ BSc Computer Science
□ MSc Computer Science
```

4. CV Sections

The following sections should use the master-data approach.

Section	What the frontend should do
Education	• Display existing education records. • Allow selection for the current CV. • Provide **+ Add Education**.
Work Experience	• Display existing experience records. • Allow selection for the current CV. • Provide **+ Add Experience**.
Skills	• Display existing skill records. • Allow selection for the current CV. • Provide **+ Add Skill**.
Certifications	• Display existing certification records. • Allow selection for the current CV. • Provide **+ Add Certification**.
Languages	• Display existing language records. • Allow selection for the current CV. • Provide **+ Add Language**.
Awards	• Display existing award records. • Allow selection for the current CV. • Provide **+ Add Award**.
Additional Information	• Display existing additional-information records. • Allow selection for the current CV. • Provide **+ Add Additional Information**.

5. Backend Endpoint Mapping

The frontend must use the **correct endpoint for each CV section**. It should **not** use one generic endpoint or one common payload for all CV sections.

CV Section	Create	List
Education	`POST /api/cvs/educations/`	`GET /api/cvs/educations/`
Work Experience	`POST /api/cvs/experiences/`	`GET /api/cvs/experiences/`
Skills	`POST /api/cvs/skills/`	`GET /api/cvs/skills/`
Certifications	`POST /api/cvs/certifications/`	`GET /api/cvs/certifications/`
Languages	`POST /api/cvs/languages/`	`GET /api/cvs/languages/`
Awards	`POST /api/cvs/awards/`	`GET /api/cvs/awards/`
Additional Information	`POST /api/cvs/additional-information/`	`GET /api/cvs/additional-information/`

Important: The exact request fields and payload structure for each endpoint should be taken from the backend API/serializer contract. Do not assume that one section's payload can be reused for another section.

6. Creating a New Master Record

For example, the user wants to add a new Education record.

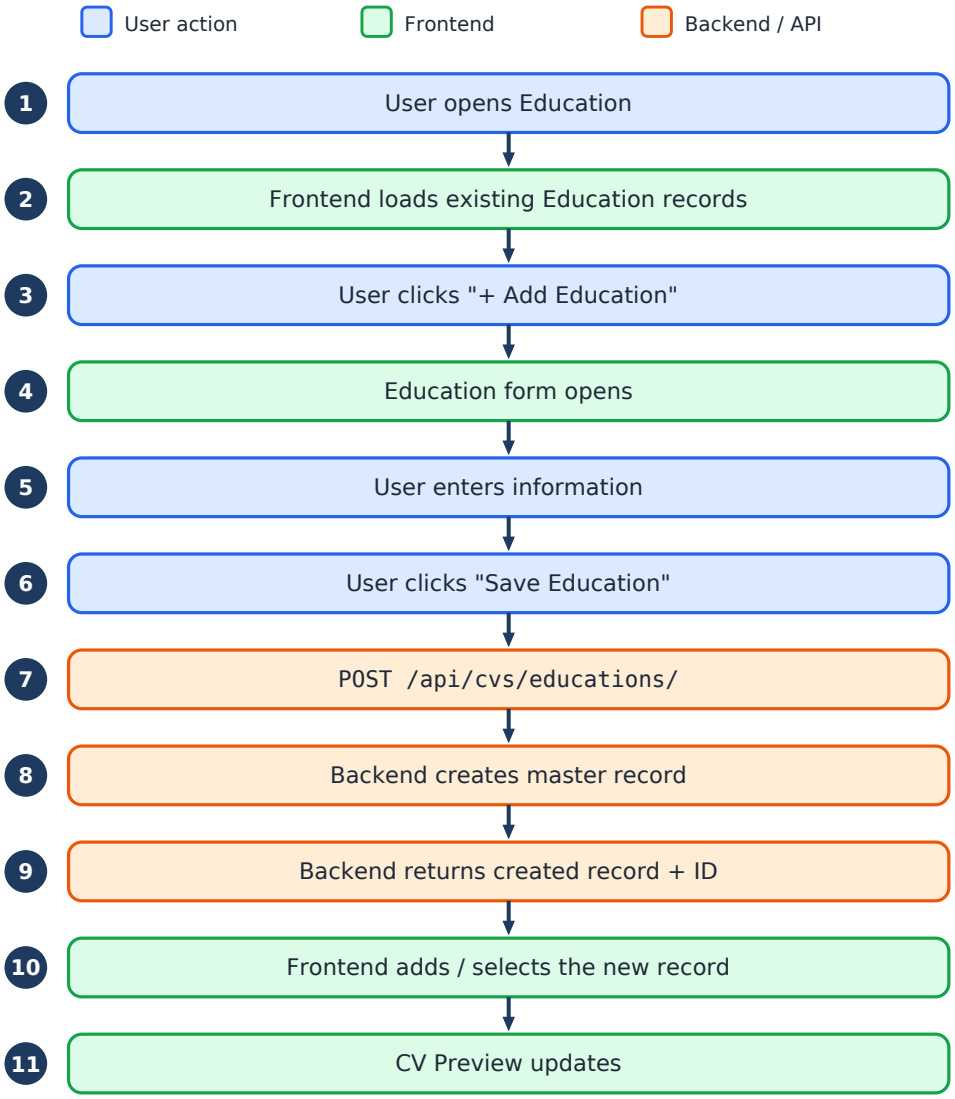


Figure 4. Frontend flow for creating a new Education master record.

The same concept applies to Experience, Skills, Certifications, Languages, Awards, and Additional Information.

7. Creating a New CV

Suppose the user already has the following master data:

- **Education:** BSc Computer Science
- **Experience:** Backend Developer, Machine Learning Intern
- **Skills:** Python, Django, AWS, SQL

- **Certifications:** AWS Certificate

The user creates a new CV for a backend developer position. Instead of typing everything again, they simply select the information relevant to this CV:

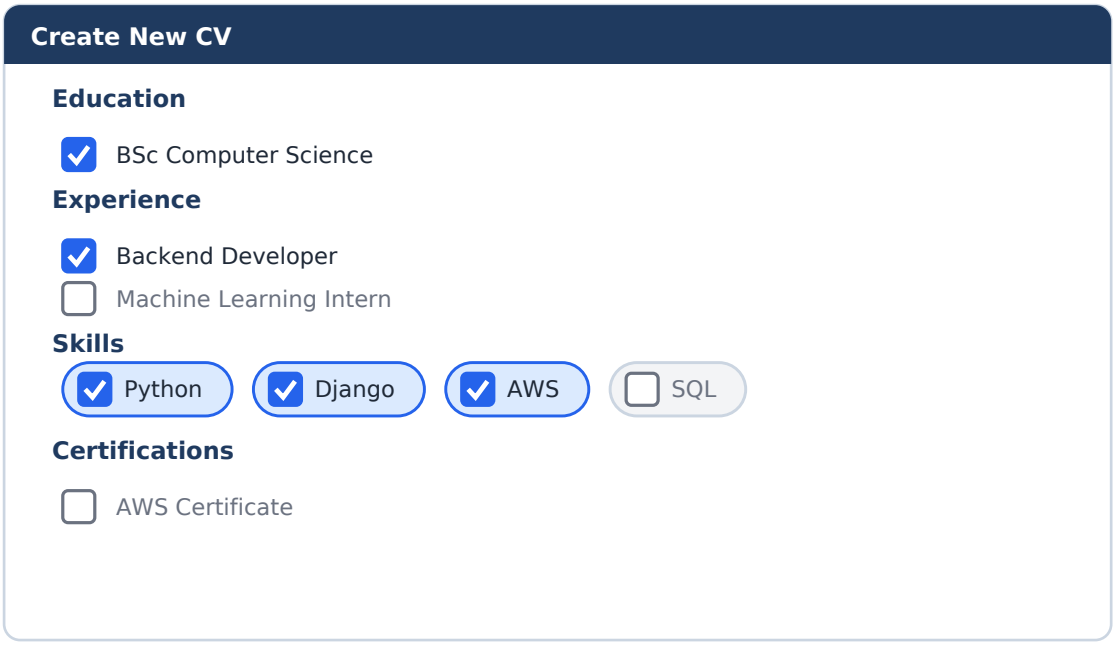


Figure 5. Selecting relevant master records for a new backend developer CV.

8. CV Data Should Reference Master Data

When an existing master record is selected, the frontend should work with the record's **ID/reference**. Conceptually:

```
{
  "template": "<template-id>",
  "professional_summary": "<summary>",
  "educations": [
    "<education-id>"
  ],
  "experiences": [
    "<experience-id>"
  ],
  "skills": [
    "<skill-id>"
  ]
}
```

This is a **conceptual example only**. The exact CV payload must follow the backend API contract.

> **Principle:** Do not duplicate the entire Education, Experience, or Skill object inside every CV. The CV should reference the user's master records.

9. Recommended “My Information” Area

A dedicated **My Information** section is recommended. This allows the user to manage their reusable information independently of their CVs.

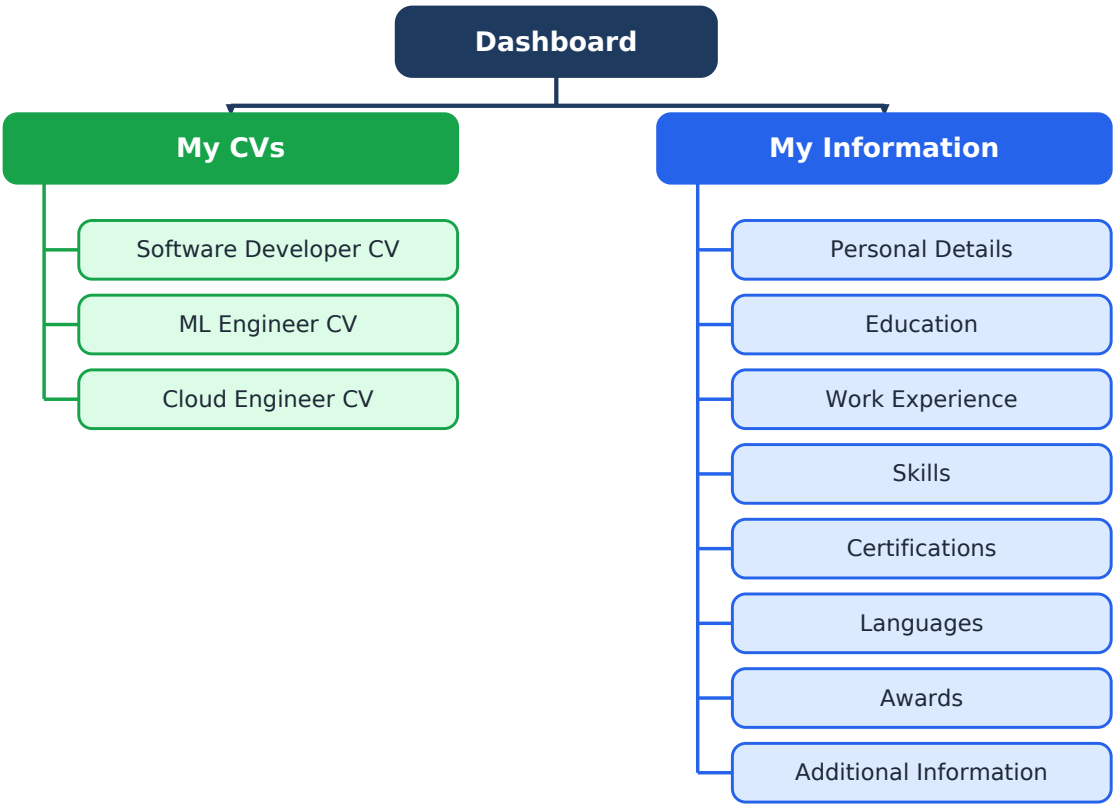


Figure 6. Suggested dashboard structure.

My Information → Education

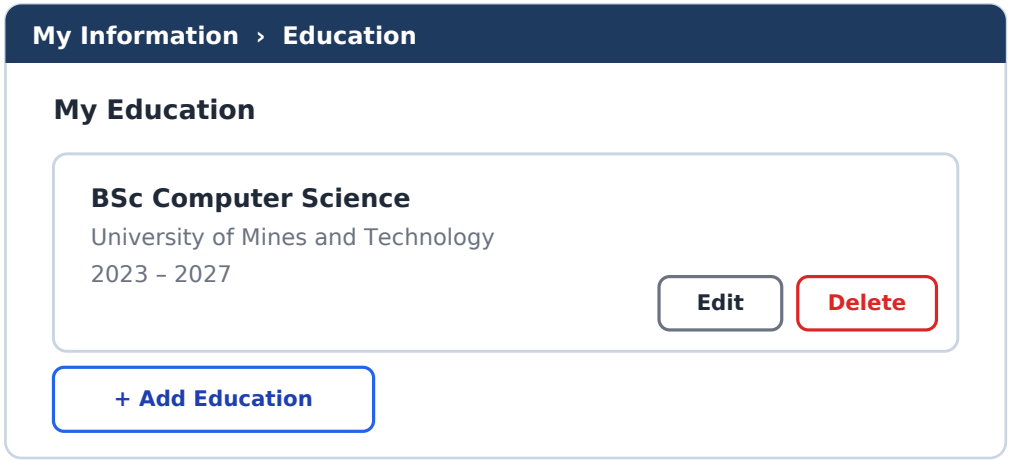


Figure 7. Managing reusable Education records outside any CV.

> However, users should **also** be able to add a new record directly from the CV Editor. They should not have to leave the editor every time they need to add something.

10. Editing Master Data

Master data may be used by multiple CVs. If the user edits *BSc Computer Science*, the change may affect all three CVs that use it. The frontend should clearly communicate this when necessary.

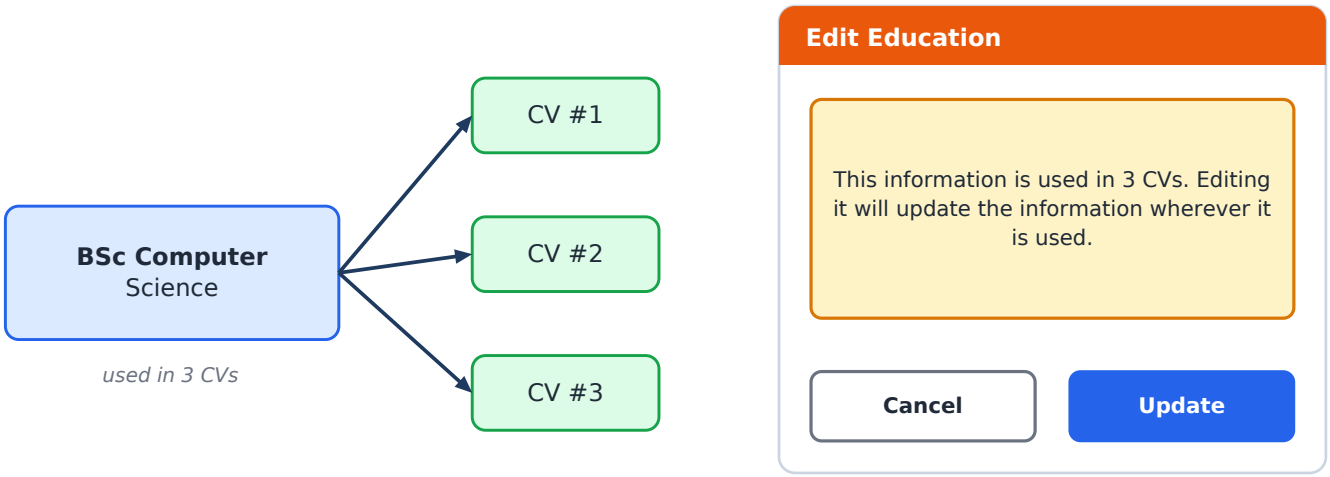


Figure 8. Editing shared master data: warn the user before the update.

11. Deleting Master Data

Deleting master data should also be handled carefully. The frontend should not silently delete a record that is in use; it should warn the user and require confirmation.

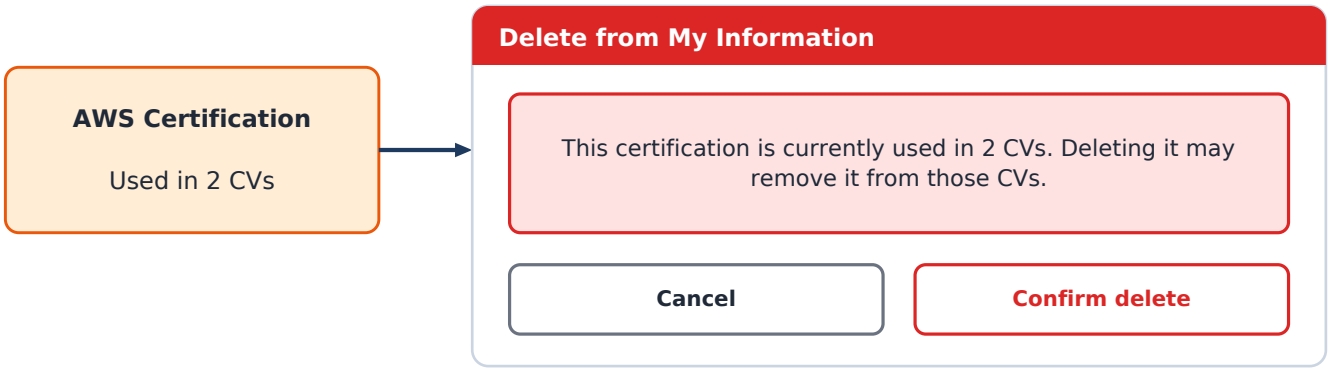


Figure 9. Warn and confirm before deleting a record that is used in CVs.

These are two **different operations** and must be treated differently:

Operation	Effect
Remove from this CV	Only removes the record from the current CV. The master record stays.
Delete from My Information	Deletes the reusable master record itself.

12. Browser Refresh and State Persistence

The frontend should not depend only on in-memory state. After a browser refresh, it should reload:

- Current CV.
- Selected template.
- Professional Summary.
- Selected Education.
- Selected Work Experience.
- Selected Skills.
- Selected Certifications.
- Selected Languages.
- Selected Awards.
- Selected Additional Information.

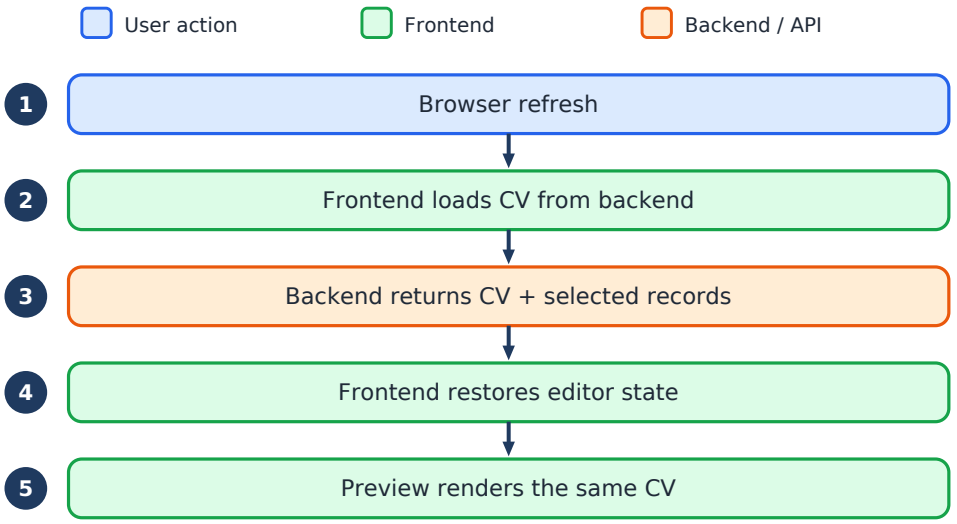


Figure 10. Restoring the editor after a refresh.

The selected template should also remain correct after refresh.

13. Authentication and Ownership

All protected requests must be made as the authenticated user. The frontend must not allow one user to access another user's CV or master data.

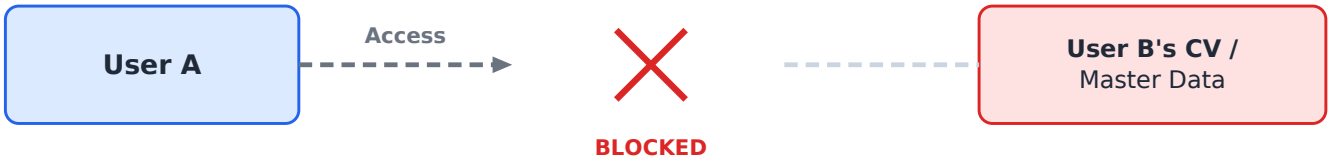


Figure 11. Ownership boundary between users.

The frontend should correctly handle authentication and authorization failures.

> Do not show “Saved successfully” when the backend actually returned an error. **The backend response should be treated as the source of truth.**

14. Error Handling

The frontend should properly handle API responses.

Status	Meaning	Frontend behaviour
400	Bad Request	Show the relevant validation/request error. Do not show a successful save.
401	Unauthorized	Handle authentication failure according to the application's authentication flow.
403	Forbidden	Inform the user that the requested operation is not permitted.
404	Not Found	Handle missing CV or master-data records appropriately.
5xx	Server Error	Show an appropriate server-error state. Do not report the operation as successfully saved.

15. CV Preview Requirements

The preview should represent the actual CV data. The frontend should ensure that:

- All selected records appear in the preview.
- Professional Summary content is fully rendered.
- Work Experience descriptions are fully rendered.
- Long content does not silently disappear.
- Long content wraps correctly within the template.
- Adding a record updates the preview.
- Removing a record updates the preview.
- Editing a record updates the preview.
- Refreshing the browser does not break the template layout.
- The selected template is preserved after refresh.

16. Recommended Frontend State Separation

The frontend should conceptually separate **master data** from **current CV data**.

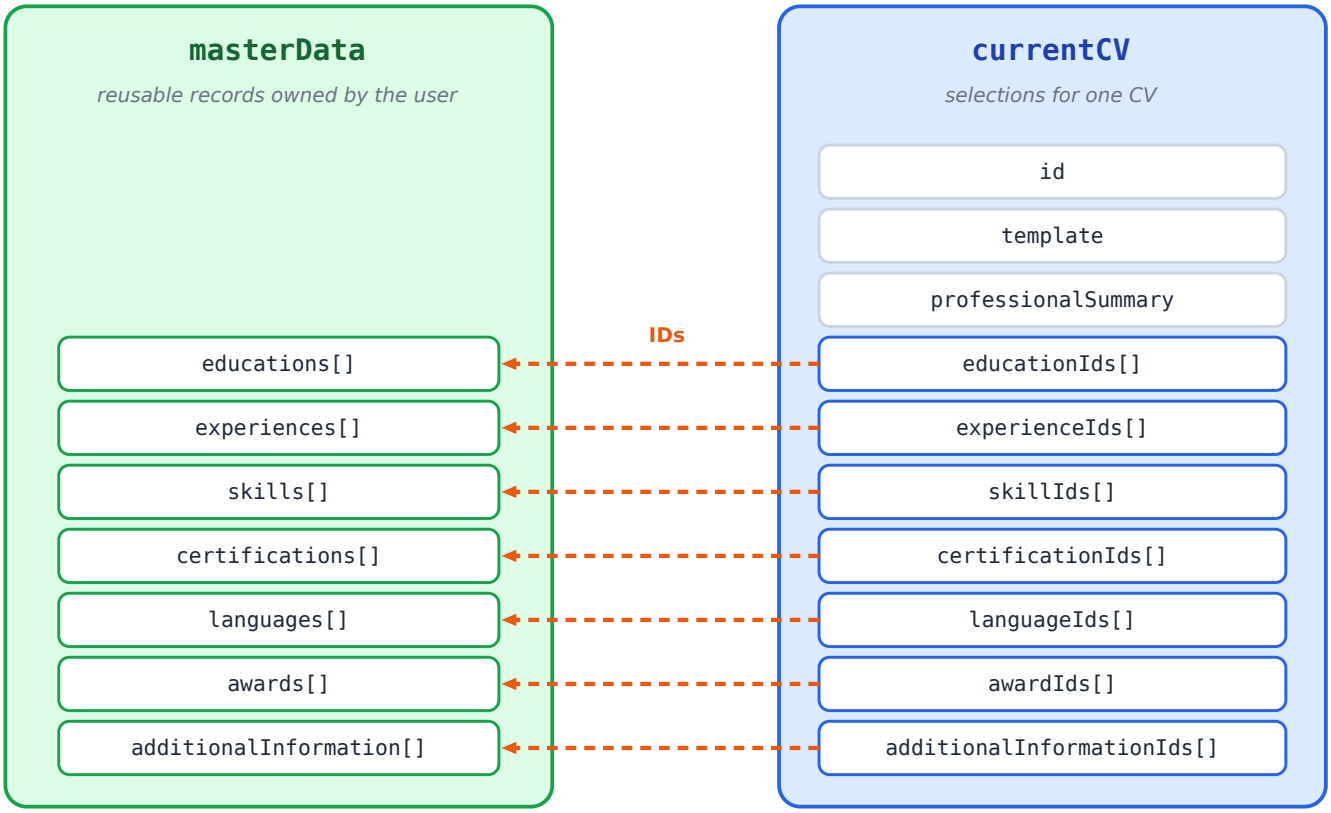


Figure 12. Master records and the current CV's selections are kept apart; the CV stores IDs only.

The exact implementation/state-management library is up to the frontend team.

> **Principle:** Reusable master records and the current CV's selections should not be treated as the same thing.

17. End-to-End Example

Consider a user who already has: Education → BSc Computer Science; Experience → Backend Developer; Skills → Python, Django, AWS; Certification → AWS Certificate. The user creates a new CV.

Step 1 — Load master data

Frontend retrieves the user's reusable information.

Step 2 — User selects information

```
Education
☑ BSc Computer Science

Experience
☑ Backend Developer

Skills
☑ Python
☑ Django
☑ AWS

Certification
☐ AWS Certificate
```

Step 3 — User adds something new

The user clicks **+ Add Certification**. The frontend calls:

```
POST /api/cvs/certifications/
```

The backend creates the master certification and returns its ID.

Step 4 — Add it to the CV

The frontend associates the returned certification with the current CV.

Step 5 — Save

The CV is saved with references to the selected master records.

Step 6 — Refresh

The browser is refreshed. The frontend retrieves the CV again and restores: Template, Professional Summary, Education, Experience, Skills, Certification.

Step 7 — Create another CV

> The user creates CV #2. The same master information is available again. **No retyping is required.**

18. Integration Acceptance Checklist

The frontend-backend integration should be considered complete when:

- [ ] Each CV section calls its corresponding backend endpoint.
- [ ] The frontend does not use one generic payload for unrelated sections.
- [ ] Existing master records can be retrieved for the authenticated user.
- [ ] Users can select existing records for a CV.
- [ ] Users can create new master records from the CV Editor.
- [ ] Newly created records can immediately be used by the current CV.
- [ ] Removing a record from a CV does not delete the master record.
- [ ] Master-data edits behave consistently wherever the record is reused.
- [ ] CV data persists after saving.
- [ ] CV data persists after browser refresh.
- [ ] The selected template persists after browser refresh.
- [ ] Backend validation errors are displayed appropriately.
- [ ] The CV Preview renders the complete selected data.
- [ ] Long text does not silently disappear from the preview.
- [ ] Users cannot access another user's CV or master data.
- [ ] API errors are not presented as successful operations.

19. Final Implementation Principle

> **The user owns reusable information. Each CV is a customized selection of that information.**

The frontend should therefore provide two complementary experiences:

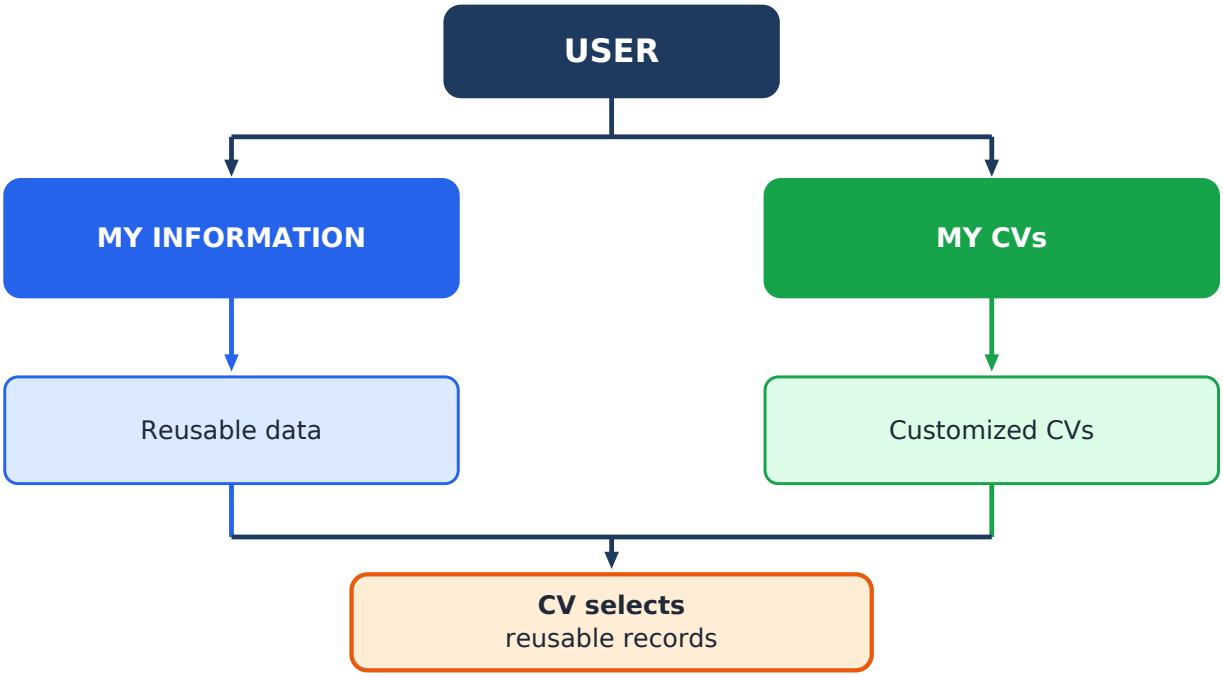


Figure 13. My Information and My CVs work together.

This approach will:

- eliminate unnecessary retyping;
- allow users to create multiple targeted CVs;
- preserve personal information independently of a CV;
- keep the frontend aligned with the backend master-data architecture; and
- make the application easier to maintain as more CV templates and sections are introduced.